Bài 2.13 – K-Means: phân nhóm khách hàng

> **Giai đoạn 2 – Machine Learning cổ điển** · Thời lượng: 1 buổi (2–3 giờ) · Hình thức mở đầu: *Thử thách có tính giờ*

🎯 Sau bài này bạn sẽ

- Hiểu và tự chạy tay thuật toán K-Means, thuật toán phân cụm phổ biến nhất.
- Chọn số cụm K bằng **phương pháp khuỷu tay** và **silhouette**.
- Thấy vì sao K-Means **bắt buộc chuẩn hóa** và cần chạy nhiều lần khởi tạo.
- Biến kết quả phân cụm thành **chân dung khách hàng** có ích cho marketing.
- Biết giới hạn của K-Means (mở đường cho DBSCAN ở bài 2.14).

---

1. Thử thách có tính giờ ⏱️ (2 phút)

Phòng Marketing đưa bạn 10 khách hàng: [thu nhập (triệu/tháng), điểm chi tiêu (1–100, càng cao càng chi mạnh)].

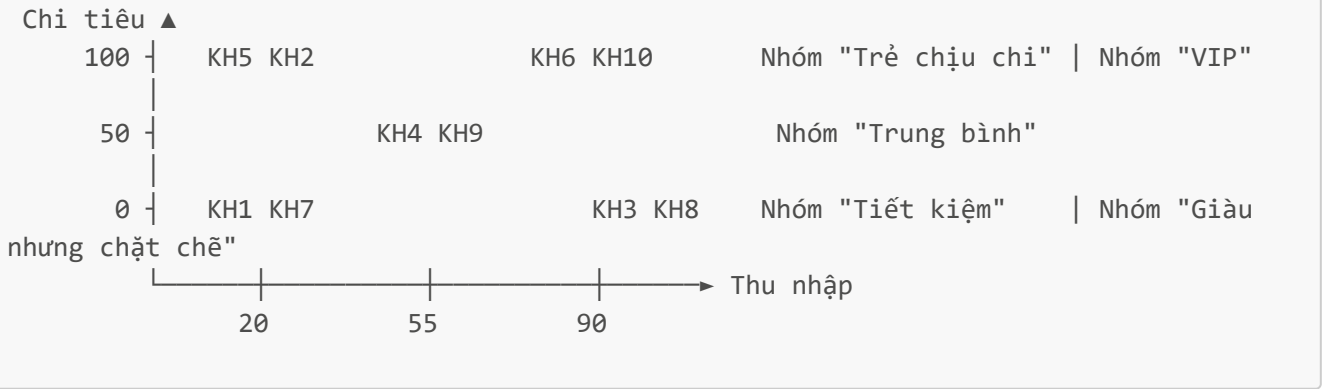
```
KH1 (20, 15)   KH2 (25, 80)   KH3 (90, 10)   KH4 (55, 50)   KH5 (22, 85)
KH6 (88, 90)   KH7 (18, 20)   KH8 (92, 15)   KH9 (52, 45)   KH10 (85, 88)
```

Đề: Trong **2 phút**, chia 10 khách thành các nhóm "giống nhau", đặt tên cho mỗi nhóm. Bấm giờ!

Gợi ý: vẽ nhanh lên giấy, trục ngang là thu nhập, trục dọc là điểm chi tiêu.

Một lời giải

> Mẹo Notion: chọn tiêu đề "Một lời giải" → `Turn into → Toggle heading 3`.



5 nhóm, mỗi nhóm cần một chiến lược riêng. Với 10 khách, bạn làm bằng mắt được. Với **1 triệu khách và 20 cột**, cần thuật toán. Đó là K-Means.

---

2. Thuật toán K-Means

2.1. Khái niệm

- **Clustering (Phân cụm):** Gom các điểm giống nhau thành nhóm, **không có nhãn** (học không giám sát, bài 2.1).
- **K-Means:** Chia dữ liệu thành **K cụm**, mỗi cụm đại diện bởi một **tâm (centroid)** là trung bình (mean) các điểm trong cụm. Mỗi điểm thuộc về cụm có tâm **gần nó nhất**.
- **Ví dụ đời thực:** Mở 3 cửa hàng trong thành phố sao cho tổng quãng đường khách đi tới cửa hàng gần nhất là nhỏ nhất.

2.2. Bốn bước

```
① Chọn ngẫu nhiên K điểm làm tâm ban đầu
② GÁN: mỗi điểm về cụm có tâm gần nhất
③ CẬP NHẬT: dời mỗi tâm về trung bình các điểm trong cụm
④ Lặp ② – ③ cho đến khi không điểm nào đổi cụm
```

2.3. Chạy tay

6 điểm: A(1, 2), B(2, 1), C(2, 3), D(8, 8), E(9, 7), F(8, 9). K = 2. Tâm ban đầu chọn **tệ** một cách cố ý: tâm 1 = A(1, 2), tâm 2 = B(2, 1).

Vòng 1 – Gán: tính khoảng cách tới 2 tâm.

- C(2, 3): tới tâm 1 = √(1 + 1) ≈ 1,41; tới tâm 2 = √(0 + 4) = 2 → **cụm 1**
- D(8, 8): tới tâm 1 = √(49 + 36) ≈ 9,22; tới tâm 2 = √(36 + 49) ≈ 9,22 → bằng nhau, chọn **cụm 1** (cụm có số thứ tự nhỏ hơn)
- E(9, 7): tới tâm 1 = √(64 + 25) ≈ 9,43; tới tâm 2 = √(49 + 36) ≈ 9,22 → **cụm 2**
- F(8, 9): tới tâm 1 = √(49 + 49) ≈ 9,90; tới tâm 2 = √(36 + 64) = 10 → **cụm 1**
- A thuộc cụm 1, B thuộc cụm 2 (khoảng cách 0 tới tâm của chính nó).

Cụm 1 = {A, C, D, F}, cụm 2 = {B, E}.

Vòng 1 – Cập nhật:

- Tâm 1 = trung bình (1, 2), (2, 3), (8, 8), (8, 9) = (19/4; 22/4) = **(4,75; 5,5)**
- Tâm 2 = trung bình (2, 1), (9, 7) = **(5,5; 4,0)**

Vòng 2: gán lại với tâm mới, nhóm trái {A, B, C} và nhóm phải {D, E, F} tách ra đúng. Tâm mới: (8,33; 8,0) và (1,67; 2,0).

Vòng 3: không điểm nào đổi cụm → **dừng**.

```python
import numpy as np

diem = np.array([[1, 2], [2, 1], [2, 3], [8, 8], [9, 7], [8, 9]], dtype=float)
tam = np.array([[1, 2], [2, 1]], dtype=float)                # tâm ban đầu: A và
B
for vong in range(1, 4):
    kc = np.linalg.norm(diem[:, None, :] - tam[None, :, :], axis=2)   # khoảng
```

```
    cách mỗi điểm tới mỗi tâm
        nhom = kc.argmin(axis=1)                                    # gán về tâm
    gần nhất
        tam = np.array([diem[nhom == k].mean(axis=0) for k in range(2)])  # dời tâm về
    trung bình cụm
        print(f"Vòng {vong}: nhóm = {nhom.tolist()} | tâm mới =
    {tam.round(2).tolist()}")
```

Kết quả mẫu:

```
Vòng 1: nhóm = [0, 1, 0, 0, 1, 0] | tâm mới = [[4.75, 5.5], [5.5, 4.0]]
Vòng 2: nhóm = [1, 1, 1, 0, 0, 0] | tâm mới = [[8.33, 8.0], [1.67, 2.0]]
Vòng 3: nhóm = [1, 1, 1, 0, 0, 0] | tâm mới = [[8.33, 8.0], [1.67, 2.0]]
```

(Code đánh số cụm từ 0.) Dù khởi đầu tệ, chỉ sau 2 vòng thuật toán đã tìm ra 2 nhóm đúng.

2.4. K-Means tối ưu cái gì?

Inertia (Quán tính) = tổng bình phương khoảng cách từ mỗi điểm tới tâm cụm của nó. Cả bước gán lẫn bước cập nhật đều **chỉ làm inertia giảm hoặc giữ nguyên**, nên thuật toán chắc chắn dừng.

```
  Inertia = Σ (khoảng cách từ điểm tới tâm cụm của nó)²
```

Inertia nhỏ = các cụm **chặt**, điểm nằm sát tâm.

---

3. Phân nhóm khách hàng

3.1. Dữ liệu (mô phỏng)

Ta **mô phỏng** 155 khách hàng của một trung tâm thương mại, với 5 nhóm "ẩn" giống phần thử thách. (Dữ liệu tự tạo, để bạn biết đáp án đúng mà kiểm tra thuật toán.)

```
import pandas as pd

rng = np.random.default_rng(7)
nhom_an = [(25, 20, 30), (25, 80, 25), (55, 50, 50), (90, 15, 25), (90, 85, 25)]
# (thu nhập TB, chi tiêu TB, số khách)
cac_nhom = [np.column_stack([rng.normal(tn, 6, n), rng.normal(ct, 7, n)]) for tn,
ct, n in nhom_an]
kh = pd.DataFrame(np.vstack(cac_nhom), columns=["thu_nhap", "diem_chi_tieu"])
kh = kh.clip(lower=[5, 1], upper=[150, 100]).round(1)      # giới hạn giá trị hợp
lý
kh["tuoi"] = rng.integers(18, 65, len(kh))
print(kh.shape)
```

Kết quả mẫu:

```
(155, 3)
```

3.2. Phân cụm

```
from sklearn.cluster import KMeans

X = kh[["thu_nhap", "diem_chi_tieu"]]
km = KMeans(n_clusters=5, n_init=10, random_state=42).fit(X)   # n_init=10: chạy
10 lần khởi tạo, giữ lần tốt nhất
kh["cum"] = km.labels_                                         # cụm của từng
khách
print(km.cluster_centers_.round(1))                            # tọa độ 5 tâm
print(round(km.inertia_, 1))
```

Kết quả mẫu:

```
[[53.8 50.3]
 [90.  86.3]
 [22.5 20. ]
 [87.9 12.9]
 [24.5 79.3]]
10542.8
```

5 tâm rất gần 5 nhóm ẩn mà ta đã tạo (55/50, 90/85, 25/20, 90/15, 25/80). K-Means tìm lại được đúng cấu trúc.

3.3. Từ cụm số đến chân dung khách hàng

Phân cụm chỉ cho ra **số hiệu cụm** (0, 1, 2...). **Ý nghĩa** do con người đặt. Đây là bước quan trọng nhất về mặt kinh doanh.

```
chan_dung = kh.groupby("cum").agg(thu_nhap=("thu_nhap", "mean"),
                                  chi_tieu=("diem_chi_tieu", "mean"),
                                  tuoi=("tuoi", "mean"),
                                  so_kh=("tuoi", "size")).round(1)
print(chan_dung)
```

Kết quả mẫu:

```
     thu_nhap  chi_tieu  tuoi  so_kh
cum
```

```
0         53.8        50.3   41.5      50
1         90.0        86.3   46.2      25
2         22.5        20.0   40.6      30
3         87.9        12.9   42.6      25
4         24.5        79.3   39.2      25
```

Cụm	Chân dung	Chiến lược gợi ý
1	**VIP**: thu nhập cao, chi mạnh	Chăm sóc riêng, ưu đãi độc quyền
3	**Giàu nhưng chặt chẽ**	Sản phẩm chất lượng cao, nhấn mạnh giá trị lâu dài
4	**Chịu chi dù thu nhập thấp**	Khuyến mãi, trả góp; cẩn thận rủi ro tín dụng
2	**Tiết kiệm**	Hàng giá tốt, combo tiết kiệm
0	**Trung bình**, nhóm đông nhất	Chương trình tích điểm đại trà

Cột `tuoi` (không dùng để phân cụm) gần như như nhau giữa các nhóm: tuổi không phải yếu tố phân biệt ở dữ liệu này.

3.4. Gán cụm cho khách mới

```
moi = pd.DataFrame([[60, 90], [100, 10]], columns=["thu_nhap", "diem_chi_tieu"])
print(km.predict(moi))          # gán về tâm gần nhất
```

Kết quả mẫu:

```
[1 3]
```

Khách thu nhập 60, chi tiêu 90 → cụm 1 (VIP). Khách thu nhập 100, chi tiêu 10 → cụm 3 (giàu nhưng chặt chẽ).

---

4. Chọn K

Ngoài đời không ai cho bạn biết có 5 nhóm. Phải tự chọn K.

4.1. Phương pháp khuỷu tay (Elbow method)

Vẽ inertia theo K. Inertia **luôn giảm** khi K tăng (K = số điểm thì inertia = 0). Tìm điểm mà đường cong **gãy gập** như khuỷu tay: sau đó thêm cụm không còn giảm được bao nhiêu.

4.2. Silhouette score (Điểm hình bóng)

- **Định nghĩa:** Với mỗi điểm, so sánh a = khoảng cách trung bình tới các điểm **cùng cụm** và b = khoảng cách trung bình tới các điểm của **cụm gần nhất khác**.

$$s = \frac{b - a}{\max(a, b)}$$

s	Nghĩa
Gần 1	Điểm nằm sâu trong cụm của mình, xa cụm khác: tốt
Gần 0	Nằm ở ranh giới giữa hai cụm
Âm	Có lẽ bị gán nhầm cụm

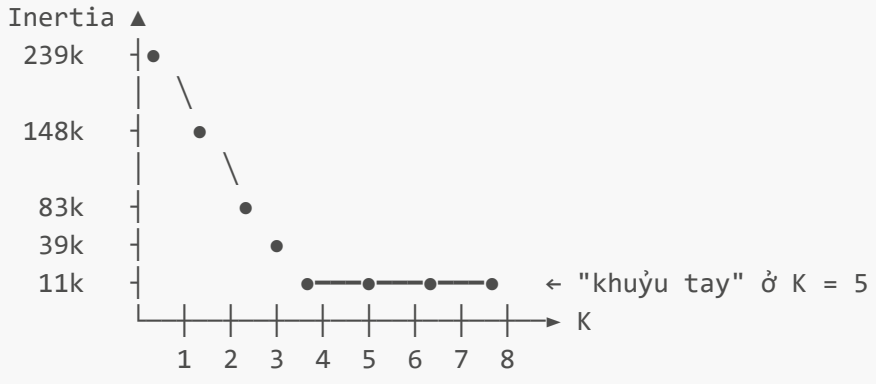
Silhouette score = trung bình s của mọi điểm. Chọn K có silhouette **cao nhất**.

```
from sklearn.metrics import silhouette_score

for k in range(1, 9):
    m = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
    s = silhouette_score(X, m.labels_) if k > 1 else float("nan")   # silhouette
cần ít nhất 2 cụm
    print(f"k = {k}: inertia = {m.inertia_:9.1f} | silhouette = {s:.3f}")
```

Kết quả mẫu:

```
k = 1: inertia =  239050.5 | silhouette = nan
k = 2: inertia =  148168.2 | silhouette = 0.364
k = 3: inertia =   83182.1 | silhouette = 0.545
k = 4: inertia =   38986.3 | silhouette = 0.649
k = 5: inertia =   10542.8 | silhouette = 0.753
k = 6: inertia =    9019.9 | silhouette = 0.633
k = 7: inertia =    8253.6 | silhouette = 0.637
k = 8: inertia =    7468.1 | silhouette = 0.568
```



Cả hai cách đều chỉ ra **K = 5**: inertia giảm mạnh tới K = 5 rồi gần như đi ngang, và silhouette đạt đỉnh 0,753 ở K = 5.

> 💡 Ngoài đời, khuỷu tay thường **không rõ** như vậy. Khi đó, hãy kết hợp: silhouette, và quan trọng nhất là **ý nghĩa kinh doanh**. Marketing có thể làm chiến dịch riêng cho 4 nhóm, nhưng không thể cho 15 nhóm.

---

5. Hai cái bẫy

5.1. Bẫy 1: không chuẩn hóa

Giả sử thu nhập được ghi bằng **đồng** (55.000.000) thay vì triệu đồng (55):

```
from sklearn.preprocessing import StandardScaler

X_dong = pd.DataFrame({"thu_nhap_dong": kh["thu_nhap"] * 1_000_000,
"diem_chi_tieu": kh["diem_chi_tieu"]})
m_sai = KMeans(n_clusters=5, n_init=10, random_state=42).fit(X_dong)
print(pd.crosstab(km.labels_, m_sai.labels_))          # so sánh với kết quả đúng
ở mục 3.2

m_dung = KMeans(n_clusters=5, n_init=10,
random_state=42).fit(StandardScaler().fit_transform(X_dong))
print(pd.crosstab(km.labels_, m_dung.labels_))
```

Kết quả mẫu:

```
col_0   0   1   2   3   4
row_0
0      24   0   0  26   0
1       0   7   0   0  18
2       0   0  30   0   0
3       0  11   0   0  14
4       0   0  25   0   0
col_0   0   1   2   3   4
row_0
0      50   0   0   0   0
1       0  25   0   0   0
2       0   0  30   0   0
3       0   0   0  25   0
4       0   0   0   0  25
```

- **Không chuẩn hóa:** khoảng cách chỉ do thu nhập (hàng chục triệu) quyết định, điểm chi tiêu bị bỏ qua. Nhóm "VIP" (hàng 1) và "giàu nhưng chặt chẽ" (hàng 3) **bị trộn lẫn**: thuật toán chỉ chia khách theo các "lát" thu nhập. Nhóm "tiết kiệm" và "chịu chi" (hàng 2, 4) bị gộp chung một cụm.
- **Có chuẩn hóa:** mỗi hàng chỉ có đúng một ô khác 0 → **khớp hoàn hảo** với kết quả đúng (chỉ khác số hiệu cụm).

K-Means dựa trên khoảng cách, giống KNN: **bắt buộc chuẩn hóa** khi các cột khác đơn vị.

5.2. Bẫy 2: khởi tạo xui

```
for seed in range(4):
    m = KMeans(n_clusters=5, n_init=1, init="random", random_state=seed).fit(X)
# chỉ 1 lần, tâm ngẫu nhiên thuần
    print(seed, round(m.inertia_, 1))
```

Kết quả mẫu:

```
0 10542.8
1 53209.6
2 10542.8
3 10542.8
```

Với seed 1, K-Means kẹt ở một lời giải **tệ gấp 5 lần** (ví dụ: 2 tâm cùng rơi vào một nhóm, còn 2 nhóm thật bị gộp làm một). K-Means chỉ đảm bảo tìm được **cực tiểu cục bộ**, giống Gradient Descent kẹt trong một hố nhỏ.

Cách chữa (mặc định của Scikit-learn):

- `init="k-means++"`: chọn tâm ban đầu **trải đều**, xa nhau.
- `n_init=10`: chạy nhiều lần với khởi tạo khác nhau, giữ lần có inertia nhỏ nhất.

---

6. Giới hạn của K-Means

Giới hạn	Hậu quả
Phải chọn K trước	Cần thử nhiều K
Giả định cụm **tròn** và **kích thước tương đương**	Thất bại với cụm hình dài, hình lưỡi liềm, lồng nhau
Mọi điểm đều bị gán vào một cụm	Outlier kéo lệch tâm; không phát hiện được điểm "lạc loài"
Chỉ dùng được cho dữ liệu số (dùng trung bình)	Cột chữ cần encoding hoặc thuật toán khác (K-Prototypes)

Giới hạn thứ hai và thứ ba sẽ được giải quyết bởi **DBSCAN** ở bài 2.14.

Ứng dụng thực tế khác: nén màu ảnh (gom hàng triệu màu thành 16 màu), gom nhóm sản phẩm, gom nhóm tài liệu theo chủ đề, chọn vị trí kho hàng.

---

7. Bài tập

1. **Chạy tay** K-Means trên 6 điểm ở mục 2.3, nhưng tâm ban đầu là A(1, 2) và D(8, 8). Cần mấy vòng để dừng?
2. Áp dụng K-Means (đã chuẩn hóa) lên 10 khách hàng ở phần thử thách (mục 1) với K = 5. Kết quả có khớp lời giải bằng mắt không?
3. Thêm cột `tuoi` vào dữ liệu phân cụm ở mục 3 (có chuẩn hóa). Silhouette tốt nhất ở K nào? Vì sao thêm cột lại làm phân cụm **kém** đi?

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

1. Với tâm A(1, 2) và D(8, 8): A, B, C gần tâm 1; D, E, F gần tâm 2 ngay từ vòng 1. Tâm mới (1,67; 2) và (8,33; 8). Vòng 2 không đổi → dừng. Chỉ cần **1 vòng cập nhật + 1 vòng kiểm tra**. Khởi tạo tốt giúp hội tụ nhanh.
2. Code:

```
kh10 = pd.DataFrame({"thu_nhap": [20, 25, 90, 55, 22, 88, 18, 92, 52, 85],
                     "diem_chi_tieu": [15, 80, 10, 50, 85, 90, 20, 15, 45, 88]},
                    index=[f"KH{i}" for i in range(1, 11)])
m10 = KMeans(n_clusters=5, n_init=10,
random_state=42).fit(StandardScaler().fit_transform(kh10))
print(pd.Series(m10.labels_, index=kh10.index).sort_values())
```

Các cặp {KH1, KH7}, {KH2, KH5}, {KH3, KH8}, {KH4, KH9}, {KH6, KH10} được gom chung, khớp với lời giải bằng mắt. 3. Code: thay `X` bằng `StandardScaler().fit_transform(kh[["thu_nhap", "diem_chi_tieu", "tuoi"]])` rồi chạy lại vòng lặp ở mục 4.2. Kết quả thật: ở K = 5, silhouette tụt từ 0,754 (2 cột, đã chuẩn hóa) xuống **0,392**; và silhouette không còn đỉnh rõ ở K = 5 nữa, mà cứ tăng dần tới K = 8 (0,484). Thuật toán bắt đầu chia nhỏ theo tuổi, tạo ra các nhóm vô nghĩa. Nguyên nhân: `tuoi` ở dữ liệu này là ngẫu nhiên, không có cấu trúc nhóm. Sau chuẩn hóa, nó có "tiếng nói" ngang các cột khác và làm nhiễu khoảng cách. Giống bài 2.7: với thuật toán dựa trên khoảng cách, thêm cột vô ích là **có hại**.

---

8. Flashcard

Mặt trước	Mặt sau
K-Means làm gì?	Chia dữ liệu thành K cụm, mỗi điểm thuộc cụm có tâm gần nhất.
Hai bước lặp của K-Means?	Gán điểm về tâm gần nhất; dời tâm về trung bình cụm.
Inertia là gì?	Tổng bình phương khoảng cách từ mỗi điểm tới tâm cụm của nó.
Phương pháp khuỷu tay?	Vẽ inertia theo K, chọn chỗ đường cong gãy gập.
Silhouette gần 1 nghĩa là?	Điểm nằm sâu trong cụm, xa cụm khác.
K-Means có cần chuẩn hóa?	Bắt buộc khi các cột khác đơn vị.
Vì sao cần `n_init` > 1?	K-Means có thể kẹt ở lời giải cục bộ tệ; chạy nhiều lần giữ lần tốt nhất.

Mặt trước	Mặt sau
k-means++ là gì?	Cách chọn tâm ban đầu trải đều, xa nhau.
K-Means giả định cụm hình gì?	Tròn, kích thước tương đương.
Sau khi phân cụm, việc quan trọng nhất?	Mô tả chân dung từng cụm và đặt tên có ý nghĩa kinh doanh.

Bài tiếp theo: 2.14 – DBSCAN.